Week 3: flow control, pseudo-code and UML activity diagrams

MIS41110 Programming for Analytics · 2026/27 · Week 3 lecture notes

These notes cover week 3's lecture, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

Where we started

Slides, pages 3, 5 and 6.

Last week's register could already do a lot. It was a two-file program, the café and a module of its sayings; it printed an aligned receipt, and its revenue survived closing time. But two things nagged us. It computed `Big order (over 10.00)?  True`, and then nothing happened: it could answer a question and could not act on the answer. And it asked one hard-coded question per product and served exactly one customer per run. Both walls fall this week. We also pick up the habit that separates a programmer from someone who just types: we draw the program before we write it.

A second look: files, and programs in two files

Recap. Week 2's two big ideas come back first: reading and writing files, and splitting a program across two files. A question on files to start:

```
log = open("data.txt")
first = log.___()
print(first)
log.close()
```

The blank must read exactly one line, so it is `readline`. The one it is easiest to confuse with is `read`, which takes everything from the bookmark to the end of the file: on a file of one line the two print the same thing, and on `menu.txt` they do not. Here is the whole idea in one picture. **`open` reads nothing.** It hands you a file object, which is two things: where the file is on disk, and a **bookmark** saying how far into it you are. Every read moves the bookmark forward. Here is `menu.txt`, all four lines:

```
>>> f = open("menu.txt")
>>> print(f.read(), end="")
espresso, single shot 2.50
latte, with oat milk 3.20
blueberry muffin, warm 2.40
tea, pot for one 2.20
>>> f.close()
```

1 `open` leaves the bookmark at 0, before the first character. 2 One `readline()` reads up to and including the line break, so the bookmark lands at 27, the start of line 2. 3 A second leaves it at 53, and 4 a third at 81. 5 `read()` jumps to 103, the end, and every read after that gives `''`.

1	`espresso, single shot 2.50` \n	**0**	`open("menu.txt")`: nothing read yet
2	`latte, with oat milk 3.20` \n	**27**	after one `readline()`
3	`blueberry muffin, warm 2.40` \n	**53**	after two
4	`tea, pot for one 2.20` \n	**81**	after three
5		**103**	`read()`: the end; the next read gives `''`

`readline` gives you one line and leaves the bookmark at the start of the next, which is why two calls give two different lines. `read` takes everything from the bookmark to the end in one go, so after it there is nothing left to read: that is why it was the wrong answer. `tell` reads the bookmark out loud and `seek` moves it; the numbers in the picture are all those two functions are.

Poll. `f = open("menu.txt")`, then `f.read(5)`, then `f.readline()`. What does the `readline` give you: the rest of line 1, the whole first line, the second line, or an error? The rest of line 1, `'sso, single shot 2.50\n'`. `read(5)` takes five characters, `'espre'`, and leaves the bookmark at 5, in the middle of espresso; `readline` reads from the bookmark to the end of that line, not from the start of the file. Python thinks in one bookmark, not in lines. `tell` shows the bookmark in between:

```
>>> f = open("menu.txt")
>>> f.read(5)
'espre'
>>> f.tell()
5
>>> f.readline()
'sso, single shot 2.50\n'
>>> f.close()
```

The `\n` at the end is week 1's line break: every line you read carries its own, and `.strip()` takes it off in section 3. And once `read` has taken everything, `seek` is how you go back:

```
>>> f = open("menu.txt")
>>> everything = f.read()
>>> f.tell()
103
>>> f.read()
''
>>> f.seek(27)
27
>>> f.readline()
'latte, with oat milk 3.20\n'
>>> f.close()
```

Writing uses the same bookmark, pointed the other way: `"w"` starts the file empty, `"a"` adds to the end, `write` takes strings only, and nothing is safely on disk until you `close`. Think of a USB key with a document open on it: the computer will not let you eject the key until the document is closed, because something is still using it. `close` is you saying you are done with the file.

The other half of the question was the two-file program. `import register_sayings` loads the file `register_sayings.py` as a module, so `cafe_w3.py` can call `register_sayings.welcome(...)`

without copying the code. `import sys` does the same with a module Python ships with. The one rule: Python looks for your module in the same folder as the program you run. I did not copy `register_sayings.py` into this week's files on purpose; it came in week 2. The simplest setup is the one I use: one folder that holds every week's café, the module and `menu.txt`, and I run every program from inside it. The practical's data file goes in a `data/` folder there, because the sheet opens it as `data/field_notes.txt`. If you keep a folder per week, each one needs its own copy of everything its program uses.

Week 2's follow-up is on the Q&A page, under Week 2.

1 · Think first: pseudo-code and activity diagrams

Slides, pages 7 to 10.

Business analytics is decisions

What can our programs do so far? Three verbs. **Receive** data, from the keyboard, a file or the arguments; **process** it, with calculations and functions; **output** it, to the screen or a file. The output of one step can be the input of the next, and you can chain them as far as you like. What is missing is doing different things depending on the data, and every business rule is exactly that: if the order is big, discount it; while stock remains, keep selling. That is **flow control**. And because flow is about paths, not lines of text, the right first tool is not code at all. It is a drawing.

Pseudo-code: the rough idea in plain words

Before you touch syntax, write the logic as a numbered mix of English and code. Tell the user whether a number is even or odd:

1. Ask the user for a number;
2. Compute the remainder of dividing it by 2 (week 1's `%`);
    - if it is 1, print "The number is odd.";
    - if it is 0, print "The number is even.";
3. Quit.

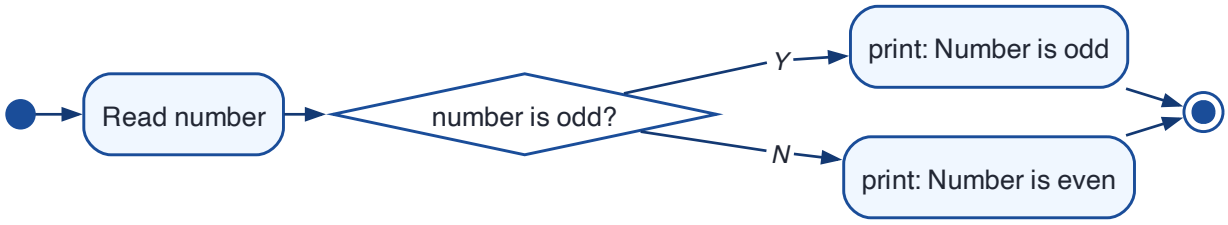
No interpreter will ever read it. It exists so you can think, and so you can explain your idea to someone else: a colleague who writes Java, R or no code at all can read it, where your Python might lose them. Research papers describe their algorithms this way too. And the part I want you to keep: when you break a problem into steps like these, you have done the real work, whoever types the code afterward. Decomposing the problem is the skill this course teaches.

The UML activity diagram: the official notation

The same logic, drawn. **UML**, the Unified Modeling Language, has exact conventions, and the exam uses them, so learn the five symbols:

- a **filled dot**: the start of the flow;
- a **round-cornered rectangle**: an action;
- a **diamond**: a decision node, and **every edge out of it gets a label**, yes and no;
- a **dot inside a circle**: the end;
- **arrows**: the direction of the flow.

Page 9 draws the even-or-odd program:



Start, read the number, hit the diamond, take one of two labeled branches, and both arrive at the end. An unlabeled edge out of a diamond loses marks.

The triptych: diagram, English, code

One program, three faithful representations, and you translate in both directions:

The diagram says	The English says	The Python says
start, then [read number]	1. Read a number	`number = input("Number? ")`
diamond: number is odd?	2. If odd, say odd;	`if int(number) % 2 == 1:`
Y edge, [print "odd"]	print "odd"	`print("Number is odd.")`
N edge, [print "even"]	else print "even"	`else:` then `print("Number is even.")`
the end node	3. Quit	the end of the program

An exam question can come in from any of the three and ask for another, so practice the translation, not the picture. The diagram breaks the problem into steps you can see, action, condition, action, and it is the one notation that comes out the same for everyone, which is the point of a *unified* language. Two people's English will be close; two people's Python can look completely different. So when you explain a program to someone else, the diagram is the version that cannot be misread.

> **In the project.** Your report carries one upper-level UML activity diagram of your program (the Project rules page lists it): this notation, drawn for the whole desk rather than one function. Draw it before you build, and your group has a plan to argue about while changing it is still cheap.

Common mistakes. A diamond whose exits carry no labels: an unlabeled edge loses marks. An arrow with no direction and a diagram with no start or end node break the notation too, and each is a sign the flow was not thought through.

Check yourself.

- **1.1** Write this as numbered pseudo-code, in the style of page 8:

```
size = input("Size? ")
if size == "large":
    print("Add 0.50.")
else:
    print("No extra charge.")
```

- **1.2** You have to explain a program to a colleague who has never written Python. Which of the three notations do you hand them, and why?

See also. Sheet 3, exercise 1. py4e chapter 3 (leave its exercise 2 for week 5: it needs `try`), or Automate the Boring Stuff chapter 2, which draws flowcharts too, with its own shapes: rounded boxes for start and end, plain boxes for steps. On the exam, use ours, from page 9.

2 · Decisions: if, elif, else

Slides, pages 11 to 16.

Poll. One word: what does an `if` let a program do? The answers centered on condition, with apply, enable and check beside it. All fair, and the word I was after is decide, or choose: an `if` lets the program take one path or another depending on the data, and the condition is the question it decides on.

if, and the cliffhanger falls

You know this one from home: "do what I say, or else". The `if` sets a condition, and the `else` catches every other possibility. In Python, at least, we know what the `else` does. The syntax is `if`, a boolean condition, a colon, then an indented block:

```
if boolean_condition:
    command          # runs only when the condition is True
    command
command              # back to unindented = always runs
```

The indented lines run only when the condition is `True`. The moment you go back to the left margin, you are out of the `if`, and that code always runs. Now last week's dead-end boolean, alive:

```
>>> total = 20.0
>>> big_order = total > 10.0
>>> big_order
True
>>> if big_order:
...     total = total * 0.90
...
>>> total
18.0
```

Two euros came off, and they came off only because the condition was `True`. With a total of eight, the block would have been skipped and nothing would have changed.

Good practice. Test a boolean directly, `if big_order:`, never `if big_order == True:`; PEP 8 says so, and the short line reads as the sentence it is. And let a range read like mathematics: Python chains comparisons, something the lectures never use, so a valid quantity is one test.

```
>>> stock = 15
>>> asked = 4
>>> 1 <= asked <= stock
True
```

Careful with types inside a condition

Ask for a number with `input` and compare it to a number, and Python teaches you something. `input` always hands back a string, and a string is never equal to an integer, even when it looks like one:

```
>>> big_number = 100
>>> answer = "100"            # what input() returns when you type 100
>>> answer == big_number
False
>>> answer >= big_number
TypeError: '>=' not supported between instances of 'str' and 'int'
>>> int(answer) >= big_number
True
```

`>=` refuses to compare the two types and stops the program. `==` is worse: it accepts the comparison and silently answers `False`, so you type 100, the program does nothing, and you debug code that has no bug, only a missing `int`. Convert once, at the input, and every comparison after it works on numbers.

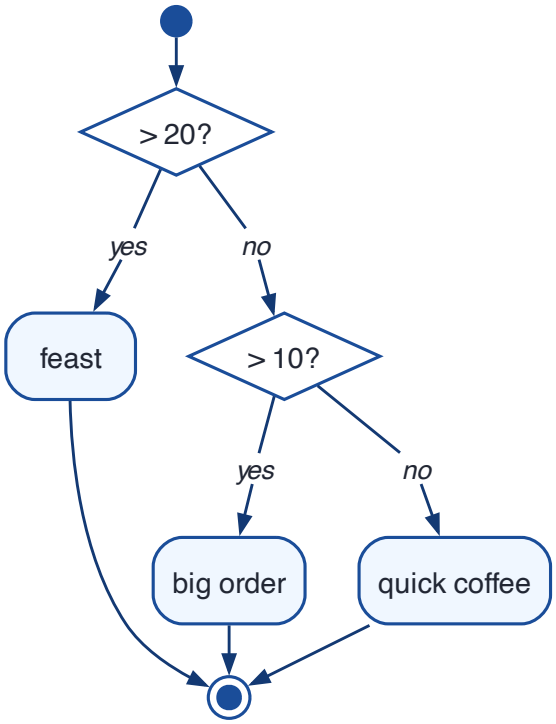
if / elif / else: the full family

Grading an order by size, with all three forms:

```
if total > 20.0:
    print("A feast!")        # if: only when True
elif total > 10.0:
    print("A big order.")    # elif: else-if, checked in order, first hit wins
else:
    print("A quick coffee.") # else: the catch-all, when nothing above matched
```

Python reads it top to bottom, and the first condition that is `True` runs; every other branch is skipped. Run it with `total = 10`: `10 > 20` is false, then `10 > 10` is false too, because greater-than is strict: ten is equal to ten, not bigger. So the `else` runs, and the quick coffee prints. With 11 the big order prints, and with 21 only the feast: 21 also passes the second test, but the first hit wins and the rest is never read. `else` never carries a condition, and `elif` can repeat as often as you need.

Drawn, it is section 1's diamond, stacked; page 12 shows the pair beside the code, and here they are with a start and an end:



The discount, built properly

Page 13. The discount in five beats, run fast, because you will run the same five on your project. **The ask**, in the owner's words: 10% off orders over 10 euros. **Decompose**: one decision, one diamond. **Design**: a function, like last week, that takes the total and hands back what to pay, so the rest of the program never has to decide anything itself. **Build**:

```
DISCOUNT_THRESHOLD = 10.0     # orders over this get a discount
DISCOUNT_RATE = 0.10          # 10% off

def apply_discount(total):
    """Happy-hour rule: a percentage off orders over the threshold."""
    if total > DISCOUNT_THRESHOLD:
        return total * (1.0 - DISCOUNT_RATE)
    else:
        return total
```

Two habits are new here. A name written all in capitals is a **constant**: Python does not enforce it, but every programmer who reads it understands "this is not meant to change while the program runs". The owner's rule is fixed, and if the owner changes their mind, say a threshold of 20, you change one line at the top and the function follows. The triple-quoted line is a **docstring**: one sentence that says what the function does, so whoever opens the file next, you included, does not have to work it out from `total * (1.0 - DISCOUNT_RATE)`. Every function from here on gets one. Being able to say exactly what a function should do is also what keeps you in charge of the code, whoever or whatever writes it.

Test at the boundary, which is where discounts always break:

```
>>> from cafe_w3 import apply_discount
>>> apply_discount(9.99)
9.99
>>> apply_discount(10.00)
10.0
>>> apply_discount(20.00)
18.0
```

Ten euros exactly gets no discount, because I wrote greater-than, not greater-or-equal. That is a decision I made on purpose, and the only way to catch a choice like that is to test the boundary. The prompt shows `10.0`; the receipt's `:.2f` is what makes it read as money.

On a line of totals, the discount starts just after 10: the open circle says 10 itself is not over the threshold. 9.99 and 10.00 sit on the near side and come back unchanged; 20.00 is over, and 10% off makes it 18.0. Nothing on this line is to scale, and its break skips most of the way to 20.00.



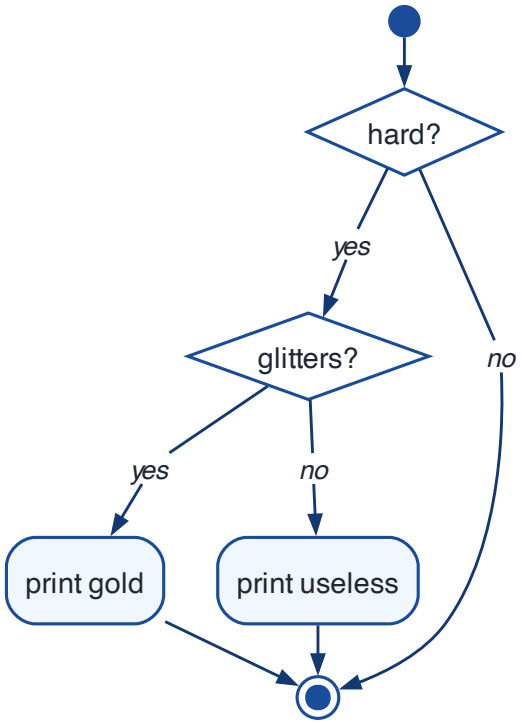
Nested ifs

An indented block can hold another `if`: decisions inside decisions. A rock: if it is hard, then if it glitters, it is gold, otherwise it is useless.

```
if hard:
    if glitters:
        print("gold")
    else:
        print("useless")
```

Read it by the indentation. Hard and glittering gives gold; hard and dull gives useless; not hard gives nothing at all, because the whole inner test lives inside the outer `if` and is never reached.

Drawn, the inner diamond sits on the outer one's yes branch, so a rock that is not hard goes straight to the end:



Here is another nested `if`, with a big number of 100 and a very big number of 1000. When the number comes from `input`, convert it once, into a name, and test the name, rather than writing `int(...)` in every condition. Here `number` already holds an int:

```
>>> big_number = 100
>>> very_big_number = 1000
>>> number = 5000
>>> if number > big_number:
...     if number > very_big_number:
...         print("This is a very big number.")
...     else:
...         print("This is a big number.")
... else:
...     print("This is a small number.")
This is a very big number.
```

With 101 it prints the big-number line, and with 1 the small-number line. Write it without the inner `else`, the second print one level up, inside the outer `if`, and 5000 prints both lines, very big and big: nothing stops the second print. The `else` at the right level is the fix, and the whole subject of the next page.

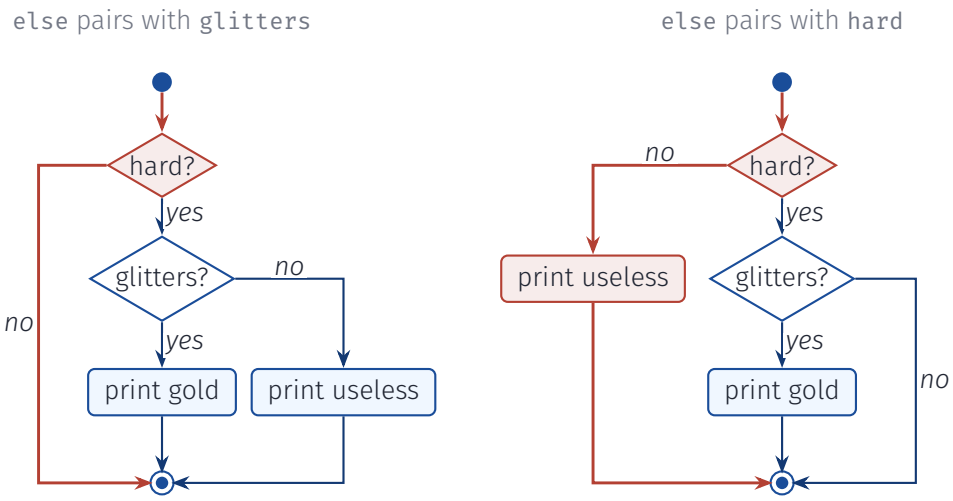
The dangling else

In many languages this one line really is ambiguous: `if hard: if glitters: gold else useless`. Does the `else` belong to the first `if` or the second? Two readings, two different programs. **Python kills the ambiguity with indentation:**

```
if hard:                          |  if hard:
    if glitters:                  |      if glitters:
        print("gold")             |          print("gold")
    else:                         |  else:
        print("useless")          |      print("useless")
# else pairs with GLITTERS        |  # else pairs with HARD
```

On the left, the `else` lines up under the inner `if`, so a rock that is hard but dull is useless. On the right, it lines up under the outer `if`, so a rock that is not hard is useless. Set `hard = False` and run both: the left version prints nothing at all, and the right one prints "useless". Same words, different indentation, different program. **The indentation is the meaning.** Nested `if`s come back in week 9, when we count the branches a program carries.

The two programs side by side, with the route lit for a rock that is not hard. On the left the `else` belongs to `glitters?`, so the no branch of `hard?` runs straight to the end and nothing prints. On the right the `else` belongs to `hard?`, so the same no branch prints useless.



Good practice. Flatten what you can. When the inner test only narrows the outer one, an `elif` chain says the same thing one level deep. Put the biggest threshold first, because the first hit wins:

```
>>> number = 5000
>>> if number > 1000:
...     print("This is a very big number.")
... elif number > 100:
...     print("This is a big number.")
... else:
...     print("This is a small number.")
...
This is a very big number.
```

Same three outcomes as the nested version, and the reader holds one condition at a time. Two levels of nesting is a warning sign; three is a function asking to be split. The café's `take_order` sits at three; week 5 splits it.

Predict, then run

Page 16's TRY IT, with `total = 12.0` and `items = 2`:

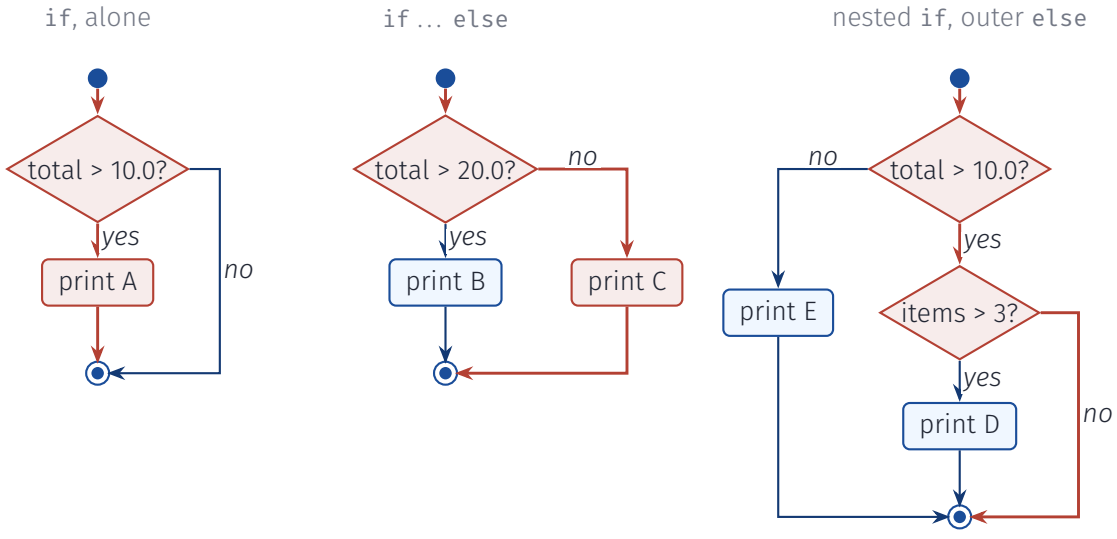
```
if total > 10.0:
    print("A")
if total > 20.0:
    print("B")
else:
    print("C")

if total > 10.0:
    if items > 3:
        print("D")
else:
    print("E")
```

Poll. What does each block print? The answer is A, then C, and nothing at all from the third block, where an E is the easy wrong guess.

Walk it. The first block is its own `if`: 12 is over 10, so A prints. The second is a separate `if`, paired with its own `else`: 12 is not over 20, so B is skipped and C prints. The third block enters the outer `if`, because 12 is over 10; inside, 2 is not more than 3, so D does not print. And the `else`? It lines up with the outer `if`, the one that was `True`, and an `else` on a `True` `if` never runs. So the program prints A and C, and the third block prints nothing at all. Move that `else` one level in, under `if items > 3:`, and the output becomes A, C, E. Same lines, one indentation, a different answer.

The three blocks as activity diagrams, with the route the run takes lit for `total = 12.0` and `items = 2`. In the first, the answer is yes and A prints. In the second, the answer is no, and its own `else` prints C. In the third, the outer answer is yes and the inner one no, so the route ends without printing. The branch to E belongs to the outer `if`, whose answer was yes, so this run never reaches it.



Common mistakes. `True` and `False` take a capital letter; written in lowercase they are just unknown names:

```
NameError: name 'false' is not defined. Did you mean: 'False'?
```

An `else` never has a condition: `else total > 2:` is `SyntaxError: expected ':'`, and so is a missing colon at the end of an `if` line. While you learn, I recommend turning off your editor's suggestions (in VS Code, switch the file's language mode to plain text): you will make more of these mistakes, and the terminal's message will teach you each one.

Check yourself.

- **2.1** With `total = 8.0` and `items = 5`, what does the TRY IT print, block by block?
- **2.2** Change one thing in `apply_discount` so that exactly 10 euros gets the discount. What does `apply_discount(10.00)` return then?
- **2.3** `code = input("Code? ")`, the user types 7, and `if code == 7:` never runs. Why not, and what are two ways to fix it?

See also. Sheet 3, exercise 1. The Function reference, Week 3: Deciding. py4e chapter 3 (leave its exercise 2 for week 5: it needs `try`).

3 · Repetition: while

Slides, pages 17 to 22.

The muffin rush: why we need while

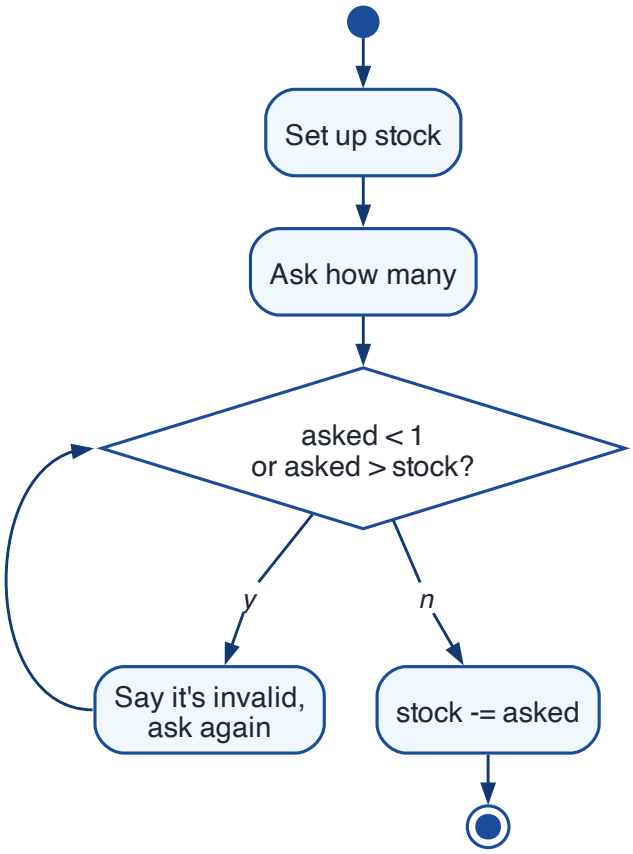
It is Saturday and we have 15 muffins. A customer wants "some". They cannot buy more than we have, or fewer than one, so we keep asking until the number makes sense. With only `if`, we nest:

```
stock = 15
asked = int(input("How many muffins? "))
if asked < 1 or asked > stock:
    print(f"You can't buy {asked} muffins.")
    asked = int(input("How many muffins? "))
    if asked < 1 or asked > stock:
        print(f"You can't buy {asked} muffins.")
        asked = int(input("How many muffins? "))
        if asked < 1 or asked > stock:
            ...
```

Horrible code, and it only survives a few wrong answers: a stubborn customer defeats the program. And without a loop, every wrong answer that ends the program means starting it again, reloading the menu, the revenue, everything, because one customer mistyped. What we want is one plain sentence: keep asking while the answer is invalid.

Pseudo-code, diagram, while

The grown-up way, in three steps. The pseudo-code: 1. set up the stock; 2. ask how many; 3. **while** the answer is invalid, keep asking; 4. update the stock. The diagram, which page 18 draws with its loop-back arrow:



That arrow going back to the diamond is the new idea, and in Python it is the `while`. Add one line at the end to show the stock, and run it:

```
stock = 15
asked = int(input("How many muffins? "))
while asked < 1 or asked > stock:
    print(f"You can't buy {asked} muffins.")
    asked = int(input("How many muffins? "))
stock -= asked
print(f"{stock} muffins left.")
```

```
How many muffins? 20
You can't buy 20 muffins.
How many muffins? 0
You can't buy 0 muffins.
How many muffins? 4
11 muffins left.
```

The nest with no end became three plain lines. One thing takes getting used to: the condition describes what you do **not** want. `asked < 1 or asked > stock` is the invalid answer, so `True` means "ask again", and the loop ends the moment the answer turns valid. Sometimes you have to turn your thinking around to write the condition: you know when to stop, and Python wants to know when to go on.

while: the syntax and the sentinel

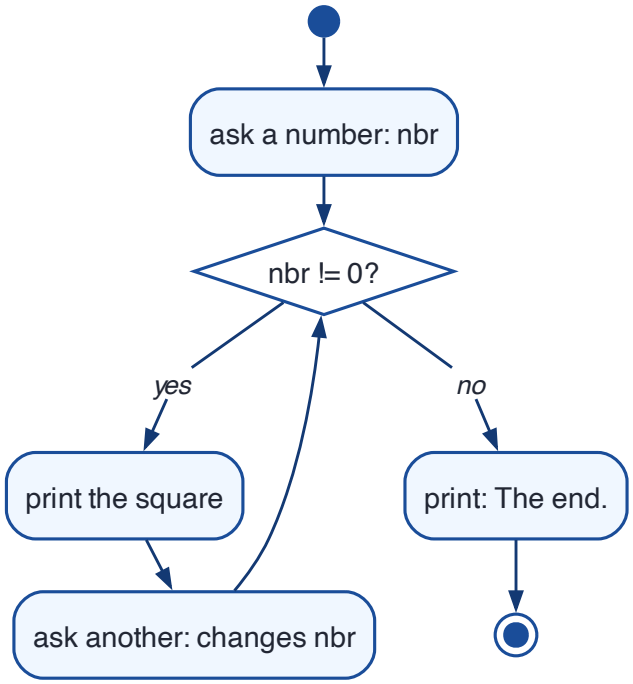
```
while boolean_condition:
    command        # repeats as long as the condition is True
    command
command            # runs once the condition turns False
```

The same shape as `if`, with one difference: after the body, Python goes back up and checks the condition again. The other shape worth memorizing repeats **until a signal**:

```
nbr = int(input("A number and I'll square it (0 to quit): "))
while nbr != 0:
    print(f"The square is {nbr * nbr}.")
    nbr = int(input("Another one (0 to quit): "))
print("The end.")
```

```
A number and I'll square it (0 to quit): 4
The square is 16.
Another one (0 to quit): 7
The square is 49.
Another one (0 to quit): 0
The end.
```

Zero is the **sentinel**, the value that means "stop". And the rule that saves you: something in the body must be able to change the condition. Here it is the second `input`. Take it out and the condition can never change. Drawn, the loop-back arrow comes back to the diamond after every pass, and the one box on the way that can change its answer is the second question:

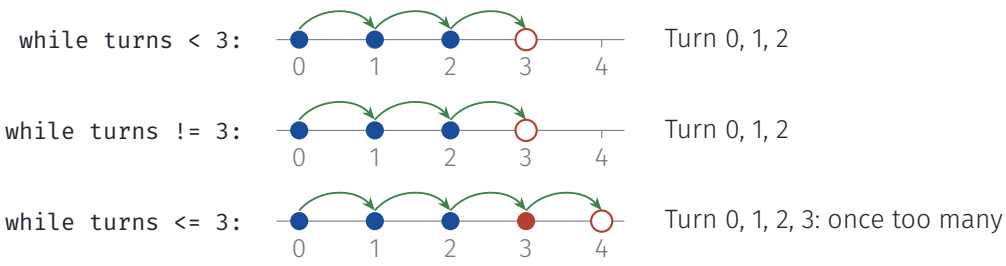


Poll. Type the condition so `while tries ___:` keeps looping until `tries` reaches 3. `< 3` is the natural answer, `!= 3` the other right one, and `<= 3` the tempting wrong one. `< 3` and `!= 3` both stop the loop the moment `tries` reaches 3, while `<= 3` runs the body once more, with `tries` at 3. Here is a loop like it, with a counter called `turns`:

```
>>> turns = 0
>>> while turns < 3:
...     print(f"Turn {turns}")
...     turns += 1
Turn 0
Turn 1
Turn 2
```

`turns += 1` is short for `turns = turns + 1`. Leave that line out and nothing in the body changes `turns`: the loop runs forever, printing `Turn 0` again and again, until Ctrl-C. With it, at 3, `3 < 3` is false and the loop stops; with `!= 3` it stops at the same moment. The two are not always the same loop, and week 4's notes open on exactly that question, walking through when `<` and `!=` part ways.

Each line follows `turns` up from 0. The body runs on the filled values, and the loop stops at the open circle, the first value whose check is false. `< 3` and `!= 3` both stop at 3, after Turn 0, 1 and 2. `<= 3` lets 3 through, and the body runs once more.



The menu loop: the shape you will reuse all term

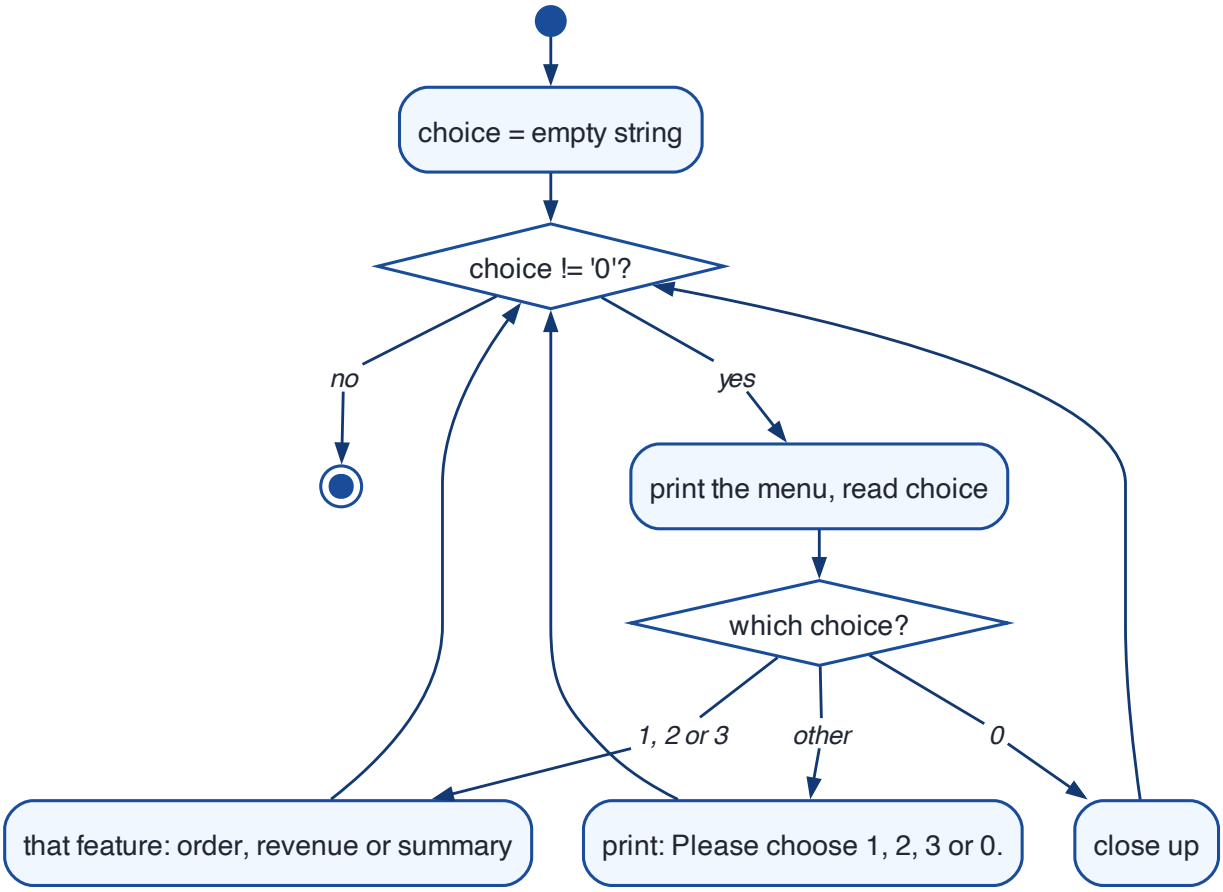
The muffin rush still ends the program once one customer is served. A real register serves many, so the whole program sits in one loop, and each feature is a small program inside it:

```
choice = ""
while choice != "0":
    print("\n[1] New order   [2] Today's revenue   [3] Day summary   [0] Close up")
    choice = input("Choose: ").strip()
    if choice == "1":
        ...          # then elif for 2, 3 and 0: one branch each
    else:
        print("Please choose 1, 2, 3 or 0.")   # junk input handled, always
```

A sentinel `while` outside, `if`/`elif`/`else` dispatch inside, and an `else` for nonsense, so junk input never crashes us. Every register, ticket machine and ATM runs this shape, and so will your project's trading desk. Look at `choice == "1"`, with quotes: `input` gives a string, and I chose, as the designer, to compare strings rather than convert. A menu choice is a label, not a number you compute with.

Drawn, the menu is one diamond that asks whether to go on and, inside the loop, a diamond with several labeled exits that sends each choice to its branch: the three features, the close, and the

`else` for anything else. Section 5 names that second shape, a multi-way decision. Every branch comes back to the top, and the loop ends when the top diamond finds `"0"`.



Two string helpers ride along. `.strip()` takes the spaces, tabs and line breaks off both ends of a string, so `"  latte \n".strip()` is `"latte"`. `input` already drops the line break of the Enter key, so on an answer `.strip()` catches the stray spaces someone types; on a line read from a file, it catches the `\n`. `.lower()` lowercases, so "Latte" and "latte" compare equal. Both are **methods**, called on a string with a dot, the same shape as `file.readline()`.

Look for the opposite of `lower` without the editor's help. A guess like `.cap()` gets `AttributeError: 'str' object has no attribute 'cap'`. A search engine offers `.capitalize()`, which turns `"the daily grind"` into `"The daily grind"`: only the first letter, not what we wanted. The documentation's list of string methods has the answer, `.upper()`, which gives `"THE DAILY GRIND"`. I would rather you work this way than ask a chatbot. The chatbot hands you the one answer; the documentation shows you the whole family on the way, `isupper`, `isdigit` and the rest, and that is how you learn what a string can do.

Good practice. Keep the menu loop thin. Each branch should hand its work to one function, `take_order()` or `day_summary()`, so the loop reads as the menu itself and each feature can be tested on its own. And every loop that waits on a person needs a way out the person can see, as the menu's 0 is.

In the project. Your desk runs on a menu loop of this shape. A text interface is one of the seven things the project's code is marked on, and the Project rules say a text menu earns full marks: a graphical interface changes nothing in the grade.

take_order: many items, one loop

Page 21. The function behind menu option 1. One customer, several items, and we do not know how many in advance, so it is a sentinel `while` with a running total:

```
def take_order():
    """Take one customer's order and return the amount they pay."""
    order_total = 0.0
    item_count = 0
    still_ordering = True
    while still_ordering:
        product = input("  Product (or 'done'): ").strip().lower()
        if product == "done":
            still_ordering = False
        else:
            price = price_of(product)
            if price == 0.0:
                print("    Sorry, we don't sell that.")
            else:
                quantity = int(input("  Quantity: "))
                order_total = order_total + price * quantity
                item_count = item_count + quantity
                print(f"    Added {quantity} x {product} ({price * quantity:.2f})")
    to_pay = apply_discount(order_total)
    print("  --- Receipt ---")
    print(f"  Items: {item_count}")
    if to_pay < order_total:
        print("  Happy-hour discount applied!")
    print(f"  Total to pay: {to_pay:.2f}")
    return to_pay
```

Three set-up variables, then a loop that runs until the customer says done. Inside, look up the price; zero means we do not sell it, so we say so and loop again; otherwise ask the quantity and add to the running total. When they are done, apply the discount, print the receipt, and return what they pay. Then run the whole register, `python3 cafe_w3.py` from the folder that holds it, and type `Ada`, `1`, `latte`, `2`, `muffin`, `1`, `done`, `3` and `0`:

```
Cashier name: Ada
Welcome to The Daily Grind, Ada!

[1] New order   [2] Today's revenue   [3] Day summary   [0] Close up
Choose: 1
  Product (or 'done'): latte
  Quantity: 2
    Added 2 x latte (6.40)
  Product (or 'done'): muffin
  Quantity: 1
    Added 1 x muffin (2.40)
  Product (or 'done'): done
  --- Receipt ---
  Items: 3
  Total to pay: 8.80

[1] New order   [2] Today's revenue   [3] Day summary   [0] Close up
Choose: 3
  Orders today:  1
  Total revenue: 8.80

[1] New order   [2] Today's revenue   [3] Day summary   [0] Close up
Choose: 0
```

```
Closed. Revenue for the day: 8.80
Thank you, come again!
```

One customer, two products, one loop, and the running total held the money. Order four lattes and seven muffins instead and the total crosses the threshold: 29.60 becomes 26.64, under the line `Happy-hour discount applied!`, the cliffhanger paid in a real run. But look at `Items: 3`. Three items, and the program cannot tell you what they were: a single number cannot remember what added up to it. Hold that pain; it is the reason next week exists.

Run the loop by hand

Slides, page 22. Tracing a loop is an exam skill: play the computer, one line at a time, and trust the table, not your gut. Same customer, two lattes, one muffin, then done:

step	input	product	price	quantity	order_total
start					`0.00`
1	latte	latte	3.20	2	`6.40`
2	muffin	muffin	2.40	1	`8.80`
3	done	done	2.40	1	`8.80`, returned

Two things in it people miss. At the start, `product`, `price` and `quantity` are empty because they do not exist yet: they are first assigned inside the loop, while `order_total` was set before it. And at step 3, `price` and `quantity` still hold 2.40 and 1, because "done" skips the whole `else`, and a variable keeps its value until something assigns it again. When the trace reaches `price_of(product)`, a name followed by parentheses is a function call, so go and find it: on a computer you search the file, on paper you scan for its `def`. It is an `if`/`elif` chain over our six products, one branch each, and latte returns 3.20. Clumsy, and exactly the clumsiness next week's dictionary erases.

> **In the project.** Your desk replays a market one day at a time, in a loop. The brief says how that loop has to behave, and back-testing it correctly is one of the seven things the code is marked on (Project rules): when you write it, trace it by hand first, as above.

Common mistakes. A loop whose body never changes its condition runs forever; press Ctrl-C in the terminal, which works on Windows, macOS and Linux alike. If a program still runs after that, end it from the task manager or the activity monitor. The opposite trap is quieter: a loop whose condition is already false runs its body zero times. Reuse `turns` for a second loop, `while turns != 3:`, right after the first, and nothing prints, because `turns` is still 3; set `turns = 0` before the second loop. And `int(input(...))` dies on an answer that is not a number, `three` or an empty Enter: week 5 fixes that.

Check yourself.

- **3.1** Trace `take_order` for this customer: `espresso`, `2`, then `bagel`, then `done`. What is `order_total` after each step, and what does the function return?
- **3.2** In the squares loop, the `input` line inside the body is deleted, and the first answer is 4. What happens?

See also. Sheet 3, exercises 2 and 3. The Function reference, Week 3: Repeating. py4e chapter 5 (leave its exercise 1 for week 5: it needs `try`), or Automate the Boring Stuff chapter 3.

4 · Element-wise: for and iterables

Slides, pages 24 to 31.

Now the second loop and the idea that goes with it: what can be looped over at all. The argument vector, a string, a range of numbers and a file are all sequences, and a `for` walks any of them one element at a time.

for: one element at a time

Remember week 2's argument vector, a row of seats: seat 0 holds the program's name, seat 1 the first argument, and so on. Last week we read them one seat at a time, by number. A `for` walks the whole row for you:

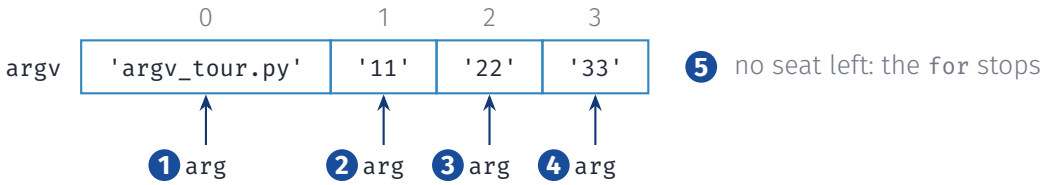
```
from sys import argv

print("Arguments to this program:", end=" ")
for arg in argv:
    print(" " + arg, end=" ")
print()
```

```
$ python3 argv_tour.py 11 22 33
Arguments to this program:  argv_tour.py  11  22  33
```

The shape is `for variable in sequence:` with an indented body, which runs once per element, the name holding each value in turn. You choose the name; `arg` could be `seat`. The split that matters: `while` is for "keep going until a condition flips", `for` is for "do this once for each element". Could you print argv with a `while` and a counter? Yes, and it would be needlessly fiddly: the `for` keeps the count for you and stops by itself after the last element. Match the loop to the job.

Each pass puts the name `arg` on the next seat's value, and every seat holds text, which is why `" " + arg` works: ❶ the program's name, `'argv_tour.py'`, then ❷ to ❹ the three arguments, `'11'`, `'22'` and `'33'`. ❺ With no seat left, the `for` stops by itself.



> **Good practice.** If you come from C or Java, resist the index loop when all you do is read. `for i in range(len(word)):` followed by `word[i]` is how those languages walk a sequence; in Python, `for letter in word:` hands you each element directly. The index loop still has its jobs, and week 4 shows them, along with `enumerate`, which hands you the position and the element together.

What counts as a sequence? Iterables

A string is a sequence of characters, one per seat, and a space is a character with a seat of its own:

```
>>> for letter in "Hello, World!":
...     print(letter + "-", end=" ")
H- e- l- l- o- ,-  - W- o- r- l- d- !-
```

Put an `if` inside and you have a filter. Page 25 prints only the vowels, e, o and o, with the test `letter == "a" or letter == "e" or ...`. Read the `or` chain the way Python does, left to right: for the letter e, the first test is false and the second is true, and one `True` is enough to make the whole condition `True`, so Python stops looking there. Loops and ifs compose, and that filter is the week's two ideas in four lines.

The same word, two jobs: in as a test

You have been writing `for letter in ...` all through this section. The word `in` has a second job: membership, the yes-or-no test "is this inside that?".

```
>>> "e" in "espresso"
True
>>> "decaf" in "double espresso, extra hot"
False
```

It answers a boolean, so it slots straight into an `if`, and the whole vowel filter collapses to one test, `letter in "aeiou"`. On strings, `in` looks for the characters in order, anywhere, and it has no idea what they mean. Write "decaf" into the middle of another word and the test says `True`; split it into "dec af" and it says `False`, because the space is a character too and the five letters no longer sit side by side. Next week the same test works on every collection we meet.

Not everything iterates

```
>>> for n in 1234:
...     print(n)
TypeError: 'int' object is not iterable
```

An integer is one value, not a sequence of digits. Week 1's conversion fixes it: loop over `str(1234)` and you get the characters 1, 2, 3 and 4, one per line. Read the message: "is not iterable" tells you exactly what went wrong.

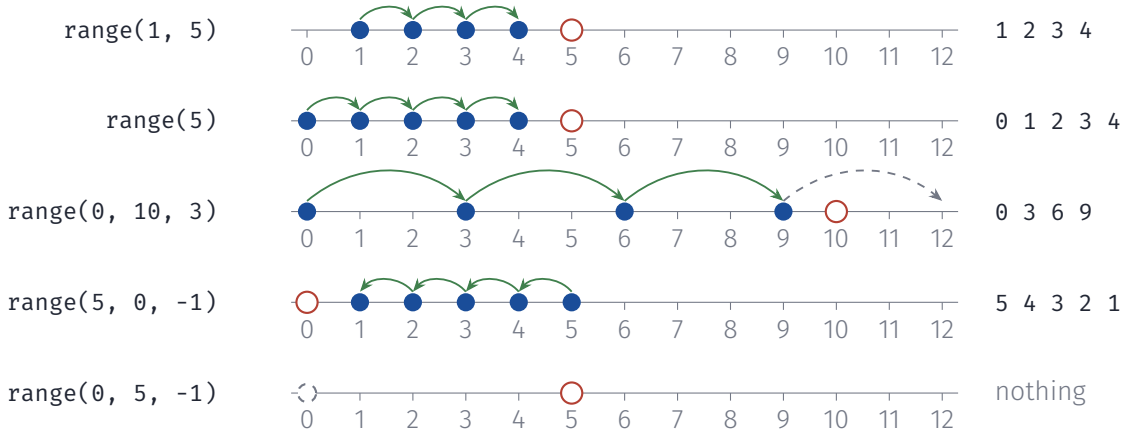
range(): sequences made to order

Four shapes, all examinable:

```
>>> for n in range(1, 5):      # start to stop-1: 1 2 3 4
>>> for n in range(5):         # 0 to stop-1:     0 1 2 3 4
>>> for n in range(0, 10, 3):  # with a step:     0 3 6 9
>>> for n in range(5, 0, -1):  # negative step:   5 4 3 2 1
```

The end value is never included, so `range(1, 5)` stops at 4: behind it is a "while smaller than 5" that stops the moment it reaches 5. `range(5)` means five numbers, starting from 0. A step moves by more than one: 0, 3, 6, 9, and 12 would pass 10, so it never comes. A negative step counts down, and stops before the end too, so `range(5, 0, -1)` gives five numbers and never reaches 0. Get the start and the end the wrong way round with a negative step, `range(0, 5, -1)`, and you get nothing at all. The everyday use is `range(n)` with a `for`, "do this n times", where you do not even look at the number.

On a number line each range starts at its first dot, hops by its step, and stops before it reaches the open circle, its stop. A step of 3 hops 0, 3, 6, 9, and the next hop would land on 12, past the stop. A step of -1 walks left and stops before 0. And `range(0, 5, -1)` gives nothing: a negative step hands out numbers only while they are above the stop, and 0 is not above 5.



The two loops convert into each other. Section 3's `turns` loop, a counter set to 0, `while turns < 3:`, and `turns += 1` at the end of the body, is `for turns in range(3):` with the print alone: `range(3)` hands out 0, 1 and 2, and the `for` stops by itself.

Matched part by part: ❶ the `while` sets the counter to 0 where `range` starts, ❷ tests `< 3` where it stops, and ❸ adds 1 where it steps. `range(3)` is `range(0, 3, 1)` written short, so all three parts hide in its defaults.

```
turns = 0 ❶
while turns < 3: ❷
    print(...)
    turns += 1 ❸
```

```
for turns in range(3):
    print(...)
```

`range(3)` is `range(0, 3, 1)` ❶ ❷ ❸

start, stop, step

Files are iterable: the day summary

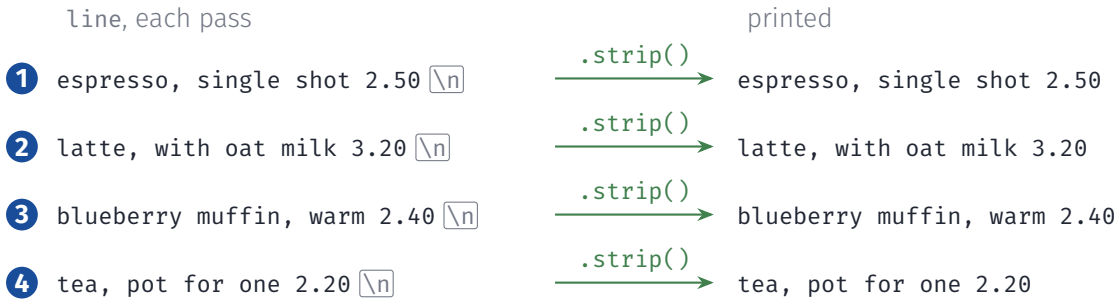
Slides, page 29. A file is a sequence of lines, so a `for` reads it as naturally as a string. The same bookmark is at work: each pass asks the file for its next line.

```
def day_summary():
    count = 0
    total = 0.0
    for line in open("receipts.txt"):
        line = line.strip()
        if line != "":
            count = count + 1
            total = total + float(line)
    print(f"  Orders today:  {count}")
    print(f"  Total revenue: {total:.2f}")
```

`for line in open(...)` is our course's one-pass read; when the loop ends, garbage collection closes the file for us (the books write `with open(...)`, which does the same job more formally). Print `menu.txt` with this loop and a bare `print(line)`, and a blank line appears after every line of the menu: the line carries its own `\n`, and `print` adds another. Either fix works: give the print an `end=""`, or print `line.strip()` instead. Do both, and the four lines glue into one long string with no breaks at all. The second fix, from your café folder:

```
>>> for line in open("menu.txt"):
...     print(line.strip())
espresso, single shot 2.50
latte, with oat milk 3.20
blueberry muffin, warm 2.40
tea, pot for one 2.20
```

Each pass, the file hands `line` its next line, line break included, and the bookmark moves on to the start of the line after it. `.strip()` takes the break off, and `print` adds its own. 1 to 4 are the four passes; when the bookmark reaches the end, the `for` stops.



Starred expressions

An iterable can be unpacked into separate print arguments with a star:

```
>>> print(range(5))
range(0, 5)
>>> print(*range(5))
0 1 2 3 4
>>> print(*"Hello, World!", sep="-")
H-e-l-l-o-,- -W-o-r-l-d-!
```

Printed plainly, a range shows you the object, not the numbers: Python does not produce the numbers until something asks for them. The star asks, and hands `print` one argument per element; `sep=` sets what goes between them, a cousin of week 1's `end=`. If you have seen a star in another language, it means something else there; in Python, in front of an iterable, it unpacks.

Your turn

Page 31's three. The divider is ten turns of `print("-", end="")` inside a `for` over `range(10)`, then one plain `print()` to end the line; you never look at `n`. For the spaces in "The Daily Grind", a first try that forgets the `if`:

```
>>> spaces = 0
>>> for character in "The Daily Grind":
...     spaces = spaces + 1
...
>>> print(spaces)
15
```

Fifteen is every character, letters included. The counter has to sit behind a test:

```
>>> spaces = 0
>>> for character in "The Daily Grind":
...     if character == " ":
...         spaces = spaces + 1
...
>>> print(spaces)
2
```

Poll. What does `for n in range(10, 0, -2): print(n)` produce? The answer is 10, 8, 6, 4 and 2, each on a line of its own: the stop value 0 is never included, and `print` ends every call with a line break, so no comma or space is printed at all. A trailing 0, or the numbers written on one line with commas, are the two easy slips.

Common mistakes. Looping over a number instead of its string, `TypeError: 'int' object is not iterable`. Expecting `range(1, 5)` to include 5. Printing a range and getting the object, `range(0, 5)`, instead of its numbers.

Check yourself.

- **4.1** Why does `for d in 2026:` fail, and what one change makes it print the digits?
- **4.2** The divider was meant to be one line of ten dashes. What does this print instead, and why?

```
for n in range(10):
    print("-")
```

- **4.3** Write this as a `while` loop that prints the same numbers:

```
for n in range(0, 10, 3):
    print(n)
```

See also. Sheet 3, exercises 4 and 5. The Function reference, Week 3: Repeating. py4e chapter 5 (leave its exercise 1 for week 5: it needs `try`).

5 · How professionals grow a program

Slides, pages 32 to 34.

Poll. Does drawing the muffin rush before coding it change what you type? It can feel like extra work. It gives the steps and breaks the problem down, and I side with that. The drawing turned a nest with no end into set up, ask, decide, and its loop-back arrow told you a `while` was coming before you typed one. Professionals keep that habit for every version of a program.

Versions, each one tested

The finished `cafe_w3.py` hides how it was made. It was not written top to bottom in one sitting. It grew in four versions, and after every version we ran it:

1. **v1, the broad idea:** the menu printed once, `if`/`elif` dispatch, no loop. Test every choice, and watch junk like "something" fall through in silence.
2. **v2, persistence:** `save_receipt` and `save_revenue`, docstrings from day one. Test again.
3. **v3, decisions:** the happy-hour discount. Test the boundaries: 9.99, 10.00, 10.01.
4. **v4, the menu loop:** wrap it in `while`, add the `else` for wrong choices. Test: can junk input kill it now?

The tests grow with the program. And you may already work this way: when the practical asks you to add a crew member with a role, typing 111 and 222 rather than real names is the natural move, because you only want to know whether the logic works. That is testing. Start rough, hard-code what you must, make it work, then refine: persistence, then docstrings, then features, then the loop. Your first version will be messy and that is normal; even the best engineers at the biggest companies start small, make mistakes and fix them, and their clean programs come out of a mess you never see. Nobody writes a looping menu program correctly in one go. And when it finally works, you get to say "I wrote that", which is a feeling no generated answer gives you.

The discipline to copy: **redraw the activity diagram before each version, then build, then test, then repeat.** Bigger diagrams use two more symbols, which you should recognize: a **multi-way decision**, one diamond with several labeled exits, and an **expand-activity** box, a round-cornered box holding a whole sub-flow of its own. The exam asks for the four basic shapes of section 1, start, action, decision and end, joined by its arrows.

> **Good practice.** Version control gives you those versions for free. Git is not part of this course; if you use it, commit each working version with a one-line message, and you can always get back to the last one that worked, or see exactly what changed since. For your group project, keep the repository private and shared with your group only: a public repository shares your code with everyone, which the integrity policy forbids.

The first-morning crash

Slides, page 33. It is the very first morning the café ever opens. Nobody has ever closed it, so there is no revenue file yet, and the program tries to read yesterday's revenue.

Poll. What does `revenue_file = open("revenue.txt")` do when there is no `revenue.txt` yet: create an empty file, give back nothing, print `0.00`, or crash? It crashes, on line one, with `FileNotFoundError`. `open` in read mode creates nothing: it goes looking, and when the file is not there, Python stops rather than guess what you meant. Only `"w"` and `"a"` create a file. Run it from a folder with no `revenue.txt` in it:

```
>>> revenue_file = open("revenue.txt")
FileNotFoundError: [Errno 2] No such file or directory: 'revenue.txt'
```

Opening it with `"w"` instead would empty yesterday's revenue every morning, which defeats the point of keeping it. Our fix this week is honest but crude, and it protects only the file we read mid-day: the café empties `receipts.txt` when it opens, with `open("receipts.txt", "w").close()`, so `day_summary` always finds it. The café does not read yesterday's revenue back yet; the week it starts is the week it needs the real fix, a program that survives missing files and bad numbers: week 5, errors and exceptions. Remember this crash; we meet it again, armed.

Pause-per-line

One more combination, because I like showing how far a few tools stretch:

```
def show_specials():
    """Display specials.txt, pausing after each line."""
    for line in open("specials.txt"):
        print(line, end="")
        dummy = input("  -- ENTER for more -- ")
```

`input` waits for the user, and here we only want the waiting: the answer is thrown away. You do not even need the name `dummy`; a bare `input("  -- ENTER for more -- ")` pauses the same way, and Python discards the text at once. Point it at `menu.txt` and run it: one line, press Enter, the next line. A `for` over a file plus an `input`: three concepts from two weeks, composed into a feature nobody taught you directly. That is fluency.

Check yourself.

- **5.1** Why does the café empty `receipts.txt` with `open("receipts.txt", "w").close()` when it opens, and why can it not do the same for `revenue.txt`?

See also. Sheet 3, exercise 6. Week 5, which fixes the crash.

6 · Three classic traps

Slides, page 35.

```
# 1. = vs ==  (Week 2's bug, now inside a condition)
if choice = "1":        # SyntaxError -- assignment, not comparison

# 2. the boundary: > or >= ?
if total > 10.0:        # exactly 10.00 gets NO discount. Intended? Be consistent.

# 3. the infinite loop
while choice != "0":
    print("menu")       # forgot to read `choice` again -> loops forever (CTRL+C to escape)
```

The first, in a real run, gives the clearest message Python has:

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

One equals sign assigns, two compare. The second is a design decision, not a bug, as long as you make it on purpose and keep to it: is 10 euros a big order or not? The third is the most common of the three, even among people who code for a living: the loop can only end when `choice` becomes "0", and nothing inside it ever reads `choice` again. When a program hangs, check trap three first: can the condition ever change?

> **Good practice.** A boundary on money needs one more care: compare cents, not floats. Three items at 3.30 come to 9.899999999999999 in floats, so a promotion "from 9.90" skips them (`round`, in the second test, arrives in week 4):
>
> ```
> >>> 3.30 * 3 >= 9.90
> False
> >>> round(3.30 * 3, 2) >= 9.90
> True
> >>> 330 * 3 >= 990
> True
> ```
>
> Round to the cent before you compare, or keep money in integer cents from the start.

Check yourself.

- **6.1** Name the trap in each. (a) The line `if total = 10:` stops the program before it starts. (b) A loop `while answer != "q":` has a body that only prints. (c) A promise of 10% off "from 10 euros" is coded as `total > 10`.

Where the café is now

Slides, pages 36 to 39.

The runnable snapshot is `cafe_w3.py`. It greets you by name, with `register_sayings` still with us, serves customers until close through the menu loop, decides (the discount acts, the cliffhanger is paid), logs every receipt, and reads the day back with a `for`. This week you received two files: `cafe_w3_start.py`, the starting point of the week's build with the rest of the café already in it, and `cafe_w3.py`, the finished program, to compare with your own. Both import `register_sayings.py`, so keep it beside them.

But feel the wall we just hit. Our order is a single running total, so we cannot list the items on the bill: a number cannot remember what made it up. And `price_of` is an `if`/`elif` chain that grows two lines longer with every product, a list of prices written into the code itself. Two different pains, one cure. Next week: lists, sets, tuples, and the dictionary that eats that chain whole.

What the exam can ask from this week, as page 37 lists it: draw an activity diagram for a described process, with the official symbols; hand-trace a `while` or a `for` with the table; find the error, the three traps and the dangling else; rewrite a `while` as a `for` or back; and state what a `range` produces. The diagram and the trace are the two to drill.

The checklist on page 39 is your revision list. Anything you cannot tick, read its section here again, then do the matching exercise on sheet 3. Self-check 3 asked the kinds of questions the exam asks, and every week builds on the one before, so a gap here costs you again next week.

Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Translate both ways from memory. Cover the code on page 18 and write it from the diagram alone; then cover the diagram and draw it from the code, with every symbol of section 1 and every diamond's edges labeled. Then draw one you have not seen drawn: the activity diagram of `take_order`, from its code on page 21, with a diamond for the loop and one for each `if`. Walk two lattes and a muffin through your drawing: it must end at 8.80.
- Run page 16's TRY IT on paper with `total = 25.0` and `items = 4`, with the diagrams in section 2 covered, write what prints, then run it to check.
- Trace `take_order` for two lattes, a croissant and a tea, in a table like page 22's, then run the café to check what it charges. Then say in one sentence why `order_total = 0.0` sits above the `while`, not inside it.
- Cover the comments beside the four `range` shapes and the number lines under them, write what each produces, then `day_summary` from page 29, from memory; then compare.
- Write a fresh one: first the pseudo-code, then `digit_sum(text)`, which returns the sum of the digits in a string of digits such as `"2026"`. Run it to check: `digit_sum("2026")` gives 10.
- In a few days, answer the check-yourself questions again without these notes, then take the week's self-check by hand, with Python closed.

Answers to check yourself

- **1.1** 1. Ask for the size; 2. if the size is large, say "Add 0.50."; otherwise, say "No extra charge."; 3. Quit. The two sayings are indented under step 2: they are its outcomes.
- **1.2** The activity diagram, or the pseudo-code. The diagram follows one notation that everyone draws the same way, so it cannot be misread; the pseudo-code needs no programming language, only plain words. The Python is the one version they cannot read.
- **2.1** Nothing from the first block, since 8 is not over 10; C from the second; and E from the third, because its `else` pairs with the outer `if total > 10.0:`, which is now `False`. So C, then E.
- **2.2** Make it `total >= DISCOUNT_THRESHOLD`. `apply_discount(10.00)` then returns `9.0`.
- **2.3** `input` returns the string `"7"`, and a string never equals the integer 7. Convert the answer, `int(code) == 7`, or compare it with a string, `code == "7"`.
- **3.1** espresso and 2: 2.50 times 2, so 5.00. bagel: `price_of` returns 0.0, so "Sorry, we don't sell that." and the total stays 5.00. done: the loop ends, 5.00 is not over 10, so the function returns 5.0 and the receipt says `Total to pay: 5.00`.
- **3.2** It never ends: `nbr` stays 4, so it prints `The square is 16.` forever, until Ctrl-C. Nothing in the body can change the condition any more.
- **4.1** An integer is one value, not a sequence: `TypeError: 'int' object is not iterable`. Loop over `str(2026)` instead.
- **4.2** Ten lines of one dash each: `print` ends every call with a line break, unless `end=""` says otherwise.
- **4.3** This prints 0, 3, 6 and 9, and the loop stops when `n` reaches 12:

```
n = 0
while n < 10:
    print(n)
    n = n + 3
```

- **5.1** So that the file `day_summary` reads mid-day always exists, starting empty each day. Opening `revenue.txt` with `"w"` would empty it every morning, and yesterday's revenue is the one thing that file is kept for.
- **6.1** (a) `=` where `==` was meant, a `SyntaxError`; (b) the infinite loop, because nothing in the body changes `answer`; (c) the boundary: "from 10 euros" includes 10, so it needs `>=`.